

---

## Table of Contents

|                                             |    |
|---------------------------------------------|----|
| .....                                       | 1  |
| maritime_yolo_review_gui_user_model.m ..... | 1  |
| ===== USER SETTINGS ===== .....             | 1  |
| ===== VALIDATE INPUT ===== .....            | 2  |
| ===== OUTPUT FOLDERS ===== .....            | 2  |
| ===== LOAD DETECTOR ===== .....             | 2  |
| ===== IMAGE LIST ===== .....                | 3  |
| ===== STATE ===== .....                     | 3  |
| ===== GUI ===== .....                       | 3  |
| ===== CALLBACKS ===== .....                 | 6  |
| ===== EXPORT ===== .....                    | 10 |
| ===== HELPERS ===== .....                   | 12 |

```
function maritime_yolo_review_gui_user_model()
```

## maritime\_yolo\_review\_gui\_user\_model.m

MATLAB Maritime Review GUI with user-defined detector mode.

Detector modes: "none" : no automatic model; user manually adds boxes. "pretrained" : use MATLAB pretrained YOLOv4 detector, e.g. "tiny-yolov4-coco". "custom" : load a custom detector from a .mat file.

Run: maritime\_yolo\_review\_gui\_user\_model

```
clear; clc; close all;
```

### ===== USER SETTINGS

=====

```
inputDir = fullfile("lars_v1.0.0_images", "val", "images");
outputDir = "LaRS_DetectionOutput_MATLAB_UserModel";

% Options: "none", "pretrained", "custom"
detectorMode = "none";
pretrainedDetectorName = "tiny-yolov4-coco";
customModelPath = fullfile("models", "maritimeDetector.mat");

confidenceThreshold = 0.25;
keepAllDetectedClasses = false;
targetClasses = ["boat", "airplane", "bird"];
manualLabels = ["boat", "ship", "submarine", "cargo_ship", "warship", ...
    "aircraft_carrier", "sailboat", "fishing_boat", "passenger_ship", ...
    "airplane", "helicopter", "bird", "buoy", "unknown"];
defaultManualLabel = "ship";
maxImagesToLoad = 200; % use Inf for all images
```

---

## ===== VALIDATE INPUT

=====

```
if ~isfolder(inputDir)
    error("Input folder not found: %s", inputDir);
end
```

## ===== OUTPUT FOLDERS

=====

```
if ~exist(outputDir, "dir"); mkdir(outputDir); end
annotatedDir = fullfile(outputDir, "reviewed_annotated");
cropDir = fullfile(outputDir, "reviewed_crops");
pixelDir = fullfile(outputDir, "reviewed_pixels");
if ~exist(annotatedDir, "dir"); mkdir(annotatedDir); end
if ~exist(cropDir, "dir"); mkdir(cropDir); end
if ~exist(pixelDir, "dir"); mkdir(pixelDir); end
```

## ===== LOAD DETECTOR

=====

```
detector = [];
detectorDescription = "None";

switch lower(detectorMode)
    case "none"
        fprintf("Detector mode: none. GUI starts with no automatic\n");
        detectorDescription = "None";

    case "pretrained"
        fprintf("Loading pretrained detector: %s\n", pretrainedDetectorName);
        detector = yolov4ObjectDetector(pretrainedDetectorName);
        detectorDescription = "Pretrained: " + pretrainedDetectorName;
        fprintf("Detector loaded.\n");

    case "custom"
        if ~isfile(customModelPath)
            error("Custom model file not found: %s", customModelPath);
        end
        fprintf("Loading custom detector: %s\n", customModelPath);
        modelData = load(customModelPath);
        if isfield(modelData, "detector")
            detector = modelData.detector;
        elseif isfield(modelData, "trainedDetector")
            detector = modelData.trainedDetector;
        elseif isfield(modelData, "maritimeDetector")
            detector = modelData.maritimeDetector;
        end
    end
```

---

```

        else
            error("Custom .mat must contain detector, trainedDetector, or
maritimeDetector.");
        end
        detectorDescription = "Custom: " + string(customModelPath);

    otherwise
        error("detectorMode must be none, pretrained, or custom.");
    end
end

```

*Detector mode: none. GUI starts with no automatic boxes.*

## ===== IMAGE LIST

=====

```

imageFiles = findImagesRecursive(inputDir);
if isempty(imageFiles)
    error("No images found under inputDir: %s", inputDir);
end
if isfinite(maxImagesToLoad)
    imageFiles = imageFiles(1:min(maxImagesToLoad, numel(imageFiles)));
end
numImages = numel(imageFiles);
fprintf("Images loaded: %d\n", numImages);

```

*Images loaded: 198*

## ===== STATE

=====

```

currentIndex = 1;
frameData = struct();
for i = 1:numImages
    frameData(i).imagePath = imageFiles(i);
    frameData(i).detected = false;
    frameData(i).reviewed = false;
    frameData(i).frameStatus = "unreviewed";
    frameData(i).boxes = struct("Label", {}, "Confidence", {}, "Position",
{}, "Source", {}, "ObjectID", {});
end
currentImage = [];
currentROIHandles = images.roi.Rectangle.empty;
selectedROIIndex = [];

```

## ===== GUI =====

```

fig = uifigure("Name", "Maritime Review GUI - User Defined Model",
"Position", [100 100 1450 850]);
mainGrid = uigridlayout(fig, [1 2]);
mainGrid.ColumnWidth = {'3x', '1x'};

```

---

```

leftPanel = uipanel(mainGrid, "Title", "Image Review");
leftGrid = uigridlayout(leftPanel, [2 1]);
leftGrid.RowHeight = {'1x', 40};
ax = uiaxes(leftGrid); ax.XTick = []; ax.YTick = []; ax.Box = "on";
statusLabel = uilabel(leftGrid, "Text", "Ready", "FontWeight", "bold");

rightPanel = uipanel(mainGrid, "Title", "Controls");
rightGrid = uigridlayout(rightPanel, [17 2]);
rightGrid.RowHeight = {30,30,30,30,30,30,30,30,30,30,30,30,30,30,30,30,'1x',30,30};
rightGrid.ColumnWidth = {'1x','1x'};

modelText = uilabel(rightGrid, "Text", "Detector: " + detectorDescription,
"WordWrap", "on");
modelText.Layout.Row = 1; modelText.Layout.Column = [1 2];
frameText = uilabel(rightGrid, "Text", "Frame:"); frameText.Layout.Row = 2;
frameText.Layout.Column = [1 2];
fileText = uilabel(rightGrid, "Text", "File:", "WordWrap", "on");
fileText.Layout.Row = 3; fileText.Layout.Column = [1 2];
statusText = uilabel(rightGrid, "Text", "Status:"); statusText.Layout.Row =
4; statusText.Layout.Column = [1 2];

prevButton = uibutton(rightGrid, "Text", "Previous", "ButtonPushedFcn",
@(~,~) goPrevious()); prevButton.Layout.Row = 5; prevButton.Layout.Column =
1;
nextButton = uibutton(rightGrid, "Text", "Next", "ButtonPushedFcn", @(~,~)
goNext()); nextButton.Layout.Row = 5; nextButton.Layout.Column = 2;
goodButton = uibutton(rightGrid, "Text", "Mark Good",
"ButtonPushedFcn", @(~,~) markFrame("good")); goodButton.Layout.Row = 6;
goodButton.Layout.Column = 1;
badButton = uibutton(rightGrid, "Text", "Mark Bad", "ButtonPushedFcn",
@(~,~) markFrame("bad")); badButton.Layout.Row = 6; badButton.Layout.Column
= 2;
addButton = uibutton(rightGrid, "Text", "Add Box", "ButtonPushedFcn", @(~,~)
addBox()); addButton.Layout.Row = 7; addButton.Layout.Column = 1;
deleteButton = uibutton(rightGrid, "Text", "Delete Selected",
"ButtonPushedFcn", @(~,~) deleteSelectedBox()); deleteButton.Layout.Row = 7;
deleteButton.Layout.Column = 2;
labelDropDown = uidropdown(rightGrid, "Items", cellstr(manualLabels),
"Value", char(defaultManualLabel)); labelDropDown.Layout.Row = 8;
labelDropDown.Layout.Column = 1;
applyLabelButton = uibutton(rightGrid, "Text", "Apply Label",
"ButtonPushedFcn", @(~,~) applySelectedLabel()); applyLabelButton.Layout.Row
= 8; applyLabelButton.Layout.Column = 2;
rerunButton = uibutton(rightGrid, "Text", "Run/Reload
Model", "ButtonPushedFcn", @(~,~) rerunDetectorOnCurrentFrame());
rerunButton.Layout.Row = 9; rerunButton.Layout.Column = 1;
clearButton = uibutton(rightGrid, "Text", "Clear Boxes",
"ButtonPushedFcn", @(~,~) clearCurrentBoxes()); clearButton.Layout.Row = 9;
clearButton.Layout.Column = 2;
saveFrameButton = uibutton(rightGrid, "Text", "Save Current
Frame", "ButtonPushedFcn", @(~,~) saveCurrentFrameOnly());
saveFrameButton.Layout.Row = 10; saveFrameButton.Layout.Column = [1 2];
exportButton = uibutton(rightGrid, "Text", "Export All Results",

```

---

```

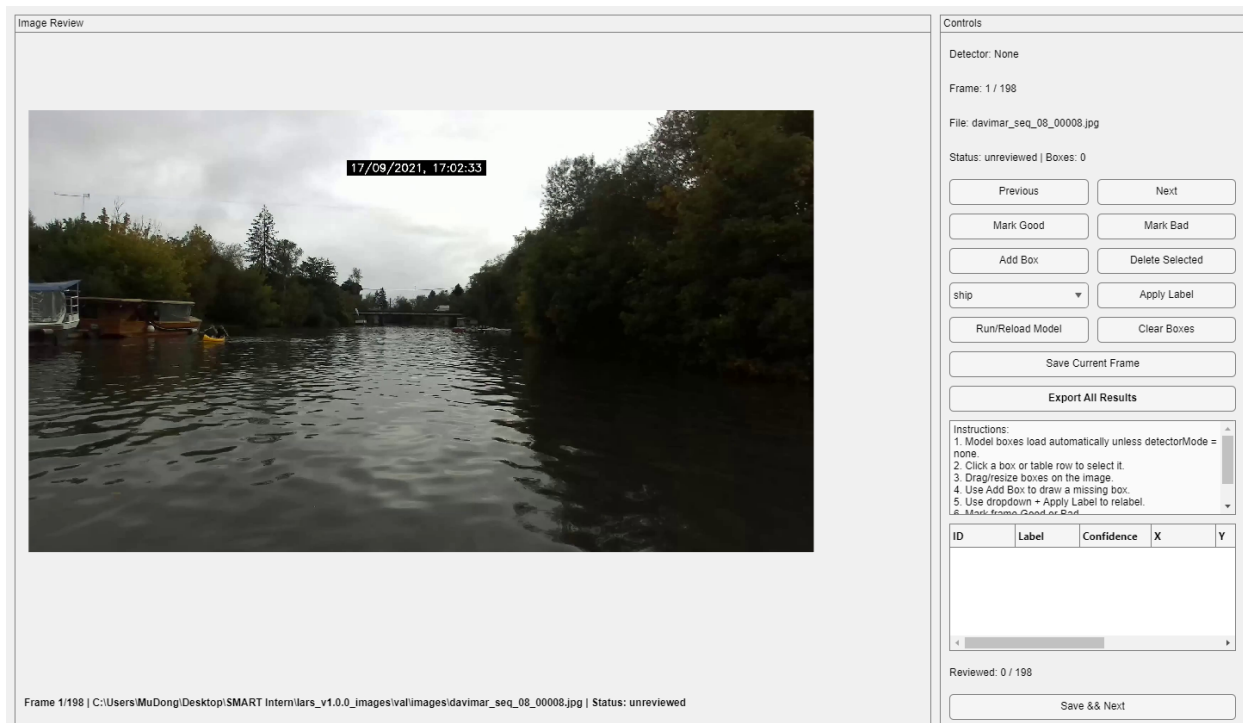
"FontWeight", "bold", "ButtonPushedFcn", @(~,~) exportAllResults());
exportButton.Layout.Row = 11; exportButton.Layout.Column = [1 2];

helpText = uitextarea(rightGrid, "Editable", "off");
helpText.Value = {'Instructions: '; '1. Model boxes load automatically unless
detectorMode = none.'; '2. Click a box or table row to select it.'; '3.
Drag/resize boxes on the image.'; '4. Use Add Box to draw a missing box.';
'5. Use dropdown + Apply Label to relabel.'; '6. Mark frame Good or Bad.';
'7. Save && Next exports current frame.'; '8. Export All Results saves all
frames.'};
helpText.Layout.Row = [12 14]; helpText.Layout.Column = [1 2];

boxTable = uitable(rightGrid);
boxTable.Layout.Row = 15; boxTable.Layout.Column = [1 2];
boxTable.ColumnName = {'ID', 'Label', 'Confidence', 'X', 'Y', 'W', 'H', 'Source'};
boxTable.ColumnEditable = [false true false false false false false false];
boxTable.CellSelectionCallback = @(src,event) tableSelectionChanged(event);
boxTable.CellEditCallback = @(src,event) tableCellEdited(event);
progressText = uilabel(rightGrid, "Text", "Progress:");
progressText.Layout.Row = 16; progressText.Layout.Column = [1 2];
saveAndNextButton = uibutton(rightGrid, "Text", "Save && Next",
"ButtonPushedFcn", @(~,~) saveAndNext()); saveAndNextButton.Layout.Row = 17;
saveAndNextButton.Layout.Column = [1 2];

loadFrame(currentIndex);

```



---

# ===== CALLBACKS

## =====

```
function loadFrame(index)
    if ~isempty(currentImage); saveCurrentROIsToMemory(); end
    currentIndex = index; selectedROIIndex = [];
    imgPath = frameData(currentIndex).imagePath;
    currentImage = imread(imgPath);
    if ~frameData(currentIndex).detected
        runDetectorForFrame(currentIndex);
    end
    refreshDisplay();
end

function runDetectorForFrame(index)
    boxes = struct("Label", {}, "Confidence", {}, "Position", {},
"Source", {}, "ObjectID", {});
    if lower(detectorMode) == "none" || isempty(detector)
        frameData(index).boxes = boxes;
        frameData(index).detected = true;
        return;
    end
    I = imread(frameData(index).imagePath);
    [bboxes, scores, labels] = detect(detector, I,
Threshold=confidenceThreshold);
    labels = string(labels);
    if ~keepAllDetectedClasses
        keepIdx = ismember(labels, targetClasses);
        bboxes = bboxes(keepIdx,:); scores = scores(keepIdx); labels =
labels(keepIdx);
    end
    for j = 1:size(bboxes,1)
        boxes(j).Label = labels(j);
        boxes(j).Confidence = scores(j);
        boxes(j).Position = bboxes(j,:);
        boxes(j).Source = detectorMode;
        boxes(j).ObjectID = j;
    end
    frameData(index).boxes = boxes;
    frameData(index).detected = true;
end

function refreshDisplay()
    deleteCurrentROIHandles(); cla(ax); imshow(currentImage, "Parent",
ax); hold(ax, "on");
    boxes = frameData(currentIndex).boxes; currentROIHandles =
images.roi.Rectangle.empty;
    for j = 1: numel(boxes)
        roi = drawrectangle(ax, "Position", boxes(j).Position, "Label",
char(boxes(j).Label), "InteractionsAllowed", "all");
        roi.UserData = j;
        addlistener(roi, "ROIDClicked", @(src,event) roiClicked(src));
    end
end
```

---

```

        addlistener(roi, "ROIMoved", @(src,event) roiMoved(src));
        currentROIHandles(j) = roi; %#ok<AGROW>
    end
    hold(ax, "off"); updateTextLabels(); refreshBoxTable();
end

function updateTextLabels()
    [~, name, ext] = fileparts(frameData(currentIndex).imagePath);
    frameText.Text = sprintf("Frame: %d / %d", currentIndex, numImages);
    fileText.Text = sprintf("File: %s%s", name, ext);
    statusText.Text = sprintf("Status: %s | Boxes: %d",
frameData(currentIndex).frameStatus, numel(frameData(currentIndex).boxes));
    reviewedArray = false(1,numImages);
    for ii = 1:numImages; reviewedArray(ii) = frameData(ii).reviewed; end
    progressText.Text = sprintf("Reviewed: %d / %d", sum(reviewedArray),
numImages);
    statusLabel.Text = sprintf("Frame %d/%d | %s |
Status: %s", currentIndex, numImages, frameData(currentIndex).imagePath,
frameData(currentIndex).frameStatus);
end

function refreshBoxTable()
    saveCurrentROIstoMemory(); boxes = frameData(currentIndex).boxes;
    if isempty(boxes); boxTable.Data = {}; return; end
    tableData = cell(numel(boxes),8);
    for j = 1:numel(boxes)
        pos = boxes(j).Position;
        tableData(j,:) = {j, char(boxes(j).Label), boxes(j).Confidence,
round(pos(1)), round(pos(2)), round(pos(3)), round(pos(4)),
char(boxes(j).Source)};
    end
    boxTable.Data = tableData;
end

function roiClicked(src); selectedROIIndex = src.UserData;
highlightSelectedROI(); end

function roiMoved(src)
    idx = src.UserData;
    if idx >= 1 && idx <= numel(frameData(currentIndex).boxes)
        frameData(currentIndex).boxes(idx).Position = src.Position;
        if ~contains(string(frameData(currentIndex).boxes(idx).Source),
"_edited")
            frameData(currentIndex).boxes(idx).Source =
string(frameData(currentIndex).boxes(idx).Source) + "_edited";
        end
    end
    refreshBoxTable();
end

function highlightSelectedROI()
    for j = 1:numel(currentROIHandles)
        if isvalid(currentROIHandles(j)); currentROIHandles(j).LineWidth
= 1; end
    end
end

```

---

---

```

        end
        if ~isempty(selectedROIIndex) && selectedROIIndex >= 1 &&
selectedROIIndex <= numel(currentROIHandles)
            if isvalid(currentROIHandles(selectedROIIndex));
currentROIHandles(selectedROIIndex).LineWidth = 4; end
        end
    end

    function tableSelectionChanged(event)
        if isempty(event.Indices); return; end
        selectedROIIndex = event.Indices(1); highlightSelectedROI();
    end

    function tableCellEdited(event)
        row = event.Indices(1); col = event.Indices(2);
        if col ~= 2; return; end
        if row >= 1 && row <= numel(frameData(currentIndex).boxes)
            frameData(currentIndex).boxes(row).Label = string(event.NewData);
            if ~contains(string(frameData(currentIndex).boxes(row).Source),
"_edited")
                frameData(currentIndex).boxes(row).Source =
string(frameData(currentIndex).boxes(row).Source) + "_edited";
            end
            if row <= numel(currentROIHandles) &&
isvalid(currentROIHandles(row)); currentROIHandles(row).Label =
char(event.NewData); end
        end
        refreshBoxTable();
    end

    function addBox()
        saveCurrentROIsToMemory(); selectedLabel =
string(labelDropDown.Value);
        roi = drawrectangle(ax, "Label", char(selectedLabel),
"InteractionsAllowed", "all");
        if isempty(roi) || ~isvalid(roi); return; end
        newIdx = numel(frameData(currentIndex).boxes) + 1;
        frameData(currentIndex).boxes(newIdx).Label = selectedLabel;
        frameData(currentIndex).boxes(newIdx).Confidence = NaN;
        frameData(currentIndex).boxes(newIdx).Position = roi.Position;
        frameData(currentIndex).boxes(newIdx).Source = "manual";
        frameData(currentIndex).boxes(newIdx).ObjectID = newIdx;
        roi.UserData = newIdx;
        addlistener(roi, "ROIClicked", @(src,event) roiClicked(src));
        addlistener(roi, "ROIMoved", @(src,event) roiMoved(src));
        currentROIHandles(newIdx) = roi; selectedROIIndex = newIdx;
%#ok<AGROW>
        refreshBoxTable(); highlightSelectedROI(); updateTextLabels();
    end

    function deleteSelectedBox()
        saveCurrentROIsToMemory();
        if isempty(selectedROIIndex); uialert(fig, "Select a bounding box
first.", "No Box Selected"); return; end

```

---



---

```

        boxes = frameData(currentIndex).boxes;
        if selectedROIIndex < 1 || selectedROIIndex > numel(boxes); return;
end
        boxes(selectedROIIndex) = []; frameData(currentIndex).boxes = boxes;
selectedROIIndex = [];
        reassignObjectIDs(); refreshDisplay();
end

function applySelectedLabel()
    if isempty(selectedROIIndex); uialert(fig, "Select a bounding box
first.", "No Box Selected"); return; end
    newLabel = string(labelDropDown.Value);
    if selectedROIIndex >= 1 && selectedROIIndex <=
numel(frameData(currentIndex).boxes)
        frameData(currentIndex).boxes(selectedROIIndex).Label = newLabel;
        if
~contains(string(frameData(currentIndex).boxes(selectedROIIndex).Source),
"_edited")
            frameData(currentIndex).boxes(selectedROIIndex).Source =
string(frameData(currentIndex).boxes(selectedROIIndex).Source) + "_edited";
        end
        if selectedROIIndex <=
numel(currentROIHandles) && isValid(currentROIHandles(selectedROIIndex));
currentROIHandles(selectedROIIndex).Label = char(newLabel); end
    end
    refreshBoxTable();
end

function clearCurrentBoxes()
    answer = uiconfirm(fig, "Remove all boxes from this frame?", "Clear
Boxes", "Options", ["Yes", "No"], "DefaultOption", "No");
    if answer ~= "Yes"; return; end
    frameData(currentIndex).boxes = struct("Label", {}, "Confidence",
{}, "Position", {}, "Source", {}, "ObjectID", {});
    selectedROIIndex = []; refreshDisplay();
end

function rerunDetectorOnCurrentFrame()
    if lower(detectorMode) == "none" || isempty(detector); uialert(fig,
"Detector mode is none. No model will run.", "No Model"); return; end
    answer = uiconfirm(fig, "Run model again and replace current
boxes?", "Run Model", "Options", ["Yes", "No"], "DefaultOption", "No");
    if answer ~= "Yes"; return; end
    frameData(currentIndex).detected = false;
runDetectorForFrame(currentIndex); selectedROIIndex = []; refreshDisplay();
end

function markFrame(status); saveCurrentROIstoMemory();
frameData(currentIndex).frameStatus = string(status);
frameData(currentIndex).reviewed = true; updateTextLabels(); end
function saveCurrentFrameOnly(); saveCurrentROIstoMemory();
frameData(currentIndex).reviewed = true; exportOneFrame(currentIndex);
uialert(fig, "Current frame exported.", "Saved"); updateTextLabels(); end
function saveAndNext(); saveCurrentROIstoMemory();

```

---

---

```

frameData(currentIndex).reviewed = true; exportOneFrame(currentIndex);
if currentIndex < numImages; loadFrame(currentIndex + 1); else;
updateTextLabels(); uialert(fig, "Last frame reached.", "Done"); end; end
    function goPrevious(); if currentIndex > 1; loadFrame(currentIndex - 1);
end; end
    function goNext(); if currentIndex < numImages; loadFrame(currentIndex +
1); end; end

    function saveCurrentROIsToMemory()
        if isempty(currentROIHandles); return; end
        boxes = frameData(currentIndex).boxes;
        for j = 1:min(numel(currentROIHandles), numel(boxes))
            if isvalid(currentROIHandles(j)); boxes(j).Position
= currentROIHandles(j).Position; boxes(j).Label =
string(currentROIHandles(j).Label); end
        end
        frameData(currentIndex).boxes = boxes;
    end

    function deleteCurrentROIHandles()
        if isempty(currentROIHandles); return; end
        for j = 1:numel(currentROIHandles); if
isvalid(currentROIHandles(j)); delete(currentROIHandles(j)); end; end
        currentROIHandles = images.roi.Rectangle.empty;
    end

    function reassignObjectIDs(); for
j = 1:numel(frameData(currentIndex).boxes);
frameData(currentIndex).boxes(j).ObjectID = j; end; end

```

## ===== EXPORT

```

function exportAllResults()
    saveCurrentROIsToMemory();
    answer = uiconfirm(fig, "Export all boxes to CSV, crops, pixels, and
annotated images?", "Export Results", "Options", ["Yes", "No"],
"DefaultOption", "Yes");
    if answer ~= "Yes"; return; end
    allResults = table();
    for idx = 1:numImages; T = exportOneFrame(idx); allResults =
[allResults; T]; %#ok<AGROW>
    end
    csvPath = fullfile(outputDir, "reviewed_detections_all.csv");
    writetable(allResults, csvPath);
    save(fullfile(outputDir, "reviewed_frameData.mat"), "frameData",
"detectorMode", "pretrainedDetectorName", "customModelPath",
"detectorDescription");
    uialert(fig, sprintf("Export complete.\n\nCSV:\n%s", csvPath),
"Export Complete");
end

```

---

```

function T = exportOneFrame(idx)
    imgPath = frameData(idx).imagePath; I = imread(imgPath); [imgH,
imgW, ~] = size(I);
    boxes = frameData(idx).boxes; [~, baseName, ~] = fileparts(imgPath);
uniqueBaseName = sprintf("%06d_%s", idx, baseName);
    T = table(); annotated = I;
    if ~isempty(boxes)
        annBoxes = zeros(numel(boxes),4); labelText =
strings(numel(boxes),1);
        for j = 1:numel(boxes)
            pos = boxes(j).Position;
            x1 = max(1, round(pos(1))); y1 = max(1, round(pos(2))); x2 =
min(imgW, round(pos(1)+pos(3)-1)); y2 = min(imgH, round(pos(2)+pos(4)-1));
            finalW = x2 - x1 + 1; finalH = y2 - y1 + 1; if finalW <= 1
|| finalH <= 1; continue; end
            annBoxes(j,:) = [x1 y1 finalW finalH];
            if isnan(boxes(j).Confidence); confText = "manual"; else;
confText = compose("%.2f", boxes(j).Confidence); end
            labelText(j) = string(boxes(j).Label) + " " + confText;
            objectCrop = I(y1:y2, x1:x2, :); [X,Y] = meshgrid(x1:x2,
y1:y2); pixelCoords = [X(:), Y(:)];
            safeLabel = makeSafeFilename(boxes(j).Label);
            cropFile = fullfile(cropDir, uniqueBaseName + "_object_" + j
+ "_" + safeLabel + ".png");
            pixelFile = fullfile(pixelDir, uniqueBaseName + "_object_" +
j + "_pixels.mat");
            imwrite(objectCrop, cropFile);
            areaPixels = finalW * finalH; sizeCategory =
classifyObjectSize(finalW, finalH); centerX = x1 + finalW/2; centerY = y1 +
finalH/2;
            normCenterX = centerX/imgW; normCenterY = centerY/imgH;
normWidth = finalW/imgW; normHeight = finalH/imgH;
            save(pixelFile, "pixelCoords", "x1", "y1", "x2", "y2",
"finalW", "finalH", "areaPixels", "centerX", "centerY", "normCenterX",
"normCenterY", "normWidth", "normHeight", "objectCrop");
            row = table(string(baseName), string(imgPath), idx, j,
string(boxes(j).Label), boxes(j).Confidence, string(boxes(j).Source),
string(frameData(idx).frameStatus), logical(frameData(idx).reviewed), x1,
y1, x2, y2, finalW, finalH, areaPixels, string(sizeCategory), centerX,
centerY, normCenterX, normCenterY, normWidth, normHeight, string(cropFile),
string(pixelFile), imgW, imgH, string(detectorDescription), 'VariableNames',
{'Frame','FullPath','FrameIndex','ObjectID','Label','Confidence','Source','Fr
ameStatus','Reviewed','X1','Y1','X2','Y2','Width','Height','AreaPixels','Size
Category','CenterX','CenterY','NormCenterX','NormCenterY','NormWidth','NormHe
ight','CropFile','PixelFile','ImageWidth','ImageHeight','DetectorDescription'
});
            T = [T; row]; %#ok<AGROW>
        end
        validRows = annBoxes(:,3) > 1 & annBoxes(:,4) > 1;
        if any(validRows); annotated = insertObjectAnnotation(I,
"rectangle", annBoxes(validRows,:), labelText(validRows)); end
    end
    imwrite(annotated, fullfile(annotatedDir, uniqueBaseName +
"_reviewed.png"));

```

---

---

```

        if isempty(T)
            T = table(string(baseName), string(imgPath), idx, NaN,
string("none"), NaN, string("none"), string(frameData(idx).frameStatus),
logical(frameData(idx).reviewed), NaN, NaN, NaN, NaN, NaN, NaN, NaN,
string("none"), NaN, NaN, NaN, NaN, NaN, NaN, string(""), string(""), imgW,
imgH, string(detectorDescription), 'VariableNames',
{'Frame','FullPath','FrameIndex','ObjectID','Label','Confidence','Source','FrameStatus','Reviewed','X1','Y1','X2','Y2','Width','Height','AreaPixels','SizeCategory','CenterX','CenterY','NormCenterX','NormCenterY','NormWidth','NormHeight','CropFile','PixelFile','ImageWidth','ImageHeight','DetectorDescription'});
        end
        writetable(T, fullfile(outputDir, uniqueBaseName + "_reviewed.csv"));
    end
end

```

## ===== HELPERS

=====

```

function files = findImagesRecursive(rootDir)
    imageExts = [".jpg", ".jpeg", ".png", ".bmp", ".tif", ".tiff"];
    listing = dir(fullfile(rootDir, "**", "**")); files = strings(0,1);
    for ii = 1:numel(listing)
        if listing(ii).isdir; continue; end
        [~,~,ext] = fileparts(listing(ii).name); ext =
lower(string(ext));
        if ismember(ext, imageExts); files(end+1,1) =
string(fullfile(listing(ii).folder, listing(ii).name)); %#ok<AGROW>
        end
    end
end

function sizeCategory = classifyObjectSize(widthPixels, heightPixels)
    areaPixels = widthPixels * heightPixels;
    if areaPixels < 32*32; sizeCategory = "tiny"; elseif areaPixels <
96*96; sizeCategory = "small"; elseif areaPixels < 224*224; sizeCategory =
"medium"; else; sizeCategory = "large"; end
end

function safeName = makeSafeFilename(label)
    safeName = string(label);
    badChars = [" ", "/", "\", ":", "*", "?", "!", "<", ">", "|"];
    for c = badChars; safeName = replace(safeName, c, "_"); end
end

end

```

*Published with MATLAB® R2024b*